

Les fonctions d'ordre supérieur en programmation fonctionnelle

Les fonctions d'ordre supérieur

1. Une fonction peut être une donnée

Dans la plupart des langages étudiés en NSI, on a l'habitude de manipuler des **nombres**, des **chaînes** ou des **listes**.

En programmation fonctionnelle, une idée essentielle est que **les fonctions elles-mêmes sont des valeurs**.

Cela signifie qu'une fonction peut :

- être stockée dans une variable,
- être passée en argument à une autre fonction,
- être renvoyée comme résultat d'une fonction.

Autrement dit, une fonction est une **expression comme les autres**.

2. Définition : fonction d'ordre supérieur

On appelle **fonction d'ordre supérieur** toute fonction qui :

- prend **au moins une fonction en paramètre**,
- ou **renvoie une fonction**,
- ou fait les deux à la fois.

Ce concept est fondamental en programmation fonctionnelle, car il permet :

- de **généraliser des traitements**,
- de **séparer le calcul du comportement**,
- et même de concevoir des programmes qui manipulent ou construisent d'autres programmes.

3. Exemple fondamental : la fonction sorted

En Python, la fonction intégrée `sorted` est un excellent exemple de fonction d'ordre supérieur.

```
sorted(iterable, *, key=None, reverse=False)
```

Le paramètre `key` permet de fournir **une fonction** qui calcule la valeur à utiliser pour comparer les éléments.

Tri par ordre alphabétique (par défaut)

```
animaux = ['veau', 'vache', 'cochon', 'anaconda', 'chat', 'chien', 'ver']
```

```
sorted(animaux)
```

Résultat :

```
['anaconda', 'chat', 'chien', 'cochon', 'vache', 'veau', 'ver']
```

Ici, Python compare directement les chaînes de caractères.

Tri selon la longueur des mots

```
sorted(animaux, key=len)
```

Résultat :

```
['ver', 'veau', 'chat', 'vache', 'chien', 'cochon', 'anaconda']
```

👉 La fonction len est **passée en paramètre** à sorted.

👉 sorted applique len à chaque élément pour effectuer le tri.

Tri selon un critère personnalisé

On peut définir sa propre fonction :

```
def deuxieme_lettre(mot):
```

```
    return mot[1]
```

```
sorted(animaux, key=deuxieme_lettre)
```

Ici, le tri se fait en comparant **la deuxième lettre** de chaque mot.

4. Les fonctions lambda (fonctions anonymes)

Pourquoi utiliser lambda ?

Dans de nombreux cas, la fonction utilisée n'est :

- ni longue,
- ni réutilisée ailleurs.

La notation lambda permet de définir **une fonction courte, sans nom**.

Syntaxe générale

lambda paramètres: expression

Exemple :

```
lambda x: 2 * x
```

Cette expression représente une fonction qui double un nombre.

```
double = lambda x: 2 * x
```

```
double(10)
```

Résultat :

20

Utilisation directe avec sorted

```
points = [(2, 4), (3, 5), (1, 1), (7, 5)]
```

```
sorted(points, key=lambda p: p[0] + p[1])
```

👉 Chaque point est trié selon sa **distance de Manhattan à l'origine**.

5. Les fonctions map, filter et reduce

Ces trois fonctions sont emblématiques de la programmation fonctionnelle.

Elles permettent de **traiter des listes sans boucle explicite**.

6. La fonction map

Rôle de map

map applique **la même fonction** à tous les éléments d'un ensemble.

```
map(fonction, iterable)
```

Exemple :

```
list(map(lambda x: 2 * x, [1, 2, 3, 4]))
```

Résultat :

```
[2, 4, 6, 8]
```

Autres exemples :

```
list(map(len, ['chat', 'vache', 'anaconda']))
```

```
list(map(lambda x: x**2, range(5)))
```

7. La fonction filter

Rôle de filter

filter permet de **sélectionner uniquement les éléments qui vérifient une condition.**

```
filter(predicat, iterable)
```

Exemple :

```
def est_pair(n):  
    return n % 2 == 0
```

```
list(filter(est_pair, range(10)))
```

Résultat :

```
[0, 2, 4, 6, 8]
```

Avec lambda :

```
list(filter(lambda m: len(m) <= 4, ['veau', 'vache', 'ver', 'anaconda']))
```

8. Combinaison de map et filter

On peut chaîner les traitements :

```
list(map(lambda x: x**2, filter(est_pair, range(6))))
```

Résultat :

```
[0, 4, 16]
```

Alternative Python : compréhension de listes

En Python, on écrit souvent :

```
[x**2 for x in range(6) if est_pair(x)]
```

👉 Les deux écritures sont équivalentes.

9. La fonction reduce

Principe

reduce permet de **réduire une liste à une seule valeur** en combinant progressivement ses éléments.

```
from functools import reduce
```

Exemple : somme des entiers de 1 à 10

```
reduce(lambda x, y: x + y, range(1, 11))
```

Exemple : produit des entiers de 1 à 5

```
reduce(lambda x, y: x * y, range(1, 6))
```

👉 reduce généralise des opérations comme :

- somme,
- produit,
- maximum,
- minimum.

10. Exemple de synthèse (approche fonctionnelle)

Problème :

Calculer la somme des carrés des entiers strictement inférieurs à 10 et multiples de 3 ou de 5.

```
reduce(  
    lambda a, b: a + b,  
    map(  
        lambda n: n*n,  
        filter(lambda n: n % 3 == 0 or n % 5 == 0, range(10))  
    )  
)
```

Cette écriture illustre :

- la **composition de fonctions**,
- l'absence de boucles,
- l'absence de variables modifiées.

11. Transparence référentielle et effets de bord

Transparence référentielle

Une fonction respecte la **transparence référentielle** si :
à mêmes entrées, elle produit toujours les mêmes sorties.

Exemple avec effet de bord (à éviter)

```
def ajouter(liste, x):
```

```
    liste.append(x)
```

```
    return liste
```

Cette fonction modifie la liste passée en paramètre.

Version fonctionnelle correcte

```
def ajouter(liste, x):
```

```
    return liste + [x]
```

👉 La liste d'origine n'est jamais modifiée.

👉 Un **nouvel objet** est créé.

12. Composition de fonctions

Plutôt que modifier une variable, on **compose** les appels :

```
ajouter(ajouter([0, 1, 2, 3], 4), 5)
```

Cette écriture est typiquement fonctionnelle.

13. Bilan à retenir

- Une **fonction d'ordre supérieur** manipule des fonctions.
- `sorted`, `map`, `filter`, `reduce` sont des fonctions d'ordre supérieur.
- La programmation fonctionnelle privilégie :
 - les expressions,
 - la composition,
 - l'absence d'effets de bord.
- On ne modifie pas les données : **on crée de nouvelles valeurs**.
- Cette approche change la manière de raisonner :
 - impératif → *comment faire*
 - fonctionnel → *quelle expression calculer*